PHASE 2 • REQUIREMENT ANALYSIS

✓ VERIFIED SPECIFICATION

Data Flow Diagram

AI-Powered Travel Planner • Smart Situational Itinerary Engine

Live URL: travel-planner-iota-plum.vercel.app **Repo:** github.com/itskian2007/travel-planner **Tech:** React + FastAPI + PostgreSQL **Team:** itskian2007

1. Level-0 Context Data Flow Diagram

The Level-0 Context Diagram defines the system boundary for the Travel Planner platform. It captures interactions between external entities (End-User Traveller and Wikipedia Commons REST API) and the central Travel Planner Processing Core.



PHASE 2 • REQUIREMENT ANALYSIS

✓ VERIFIED SPECIFICATION

Data Flow Diagram

AI-Powered Travel Planner • Smart Situational Itinerary Engine

Live URL: travel-planner-iota-plum.vercel.app **Repo:** github.com/itskian2007/travel-planner **Tech:** React + FastAPI + PostgreSQL **Team:** itskian2007

2. Level-1 Subsystem Data Flow Architecture

P1: Auth & Session — Process 1.0

Inputs: Email, bcrypt hash, password.
Outputs: Signed JWT bearer token.
Data Store: DS-1 (Users Table).

P2: Filter Engine — Process 2.0

Inputs: Mood, Diet, Budget, Persona.
Outputs: Filtered candidate destinations.
Data Store: DS-2 (Destinations Data).

P3: Itinerary Gen — Process 3.0

Inputs: Destination ID, budget factor.
Outputs: 10-day structured plan.
Data Store: DS-3 (Itineraries DB).

P4: Media Pipeline — Process 4.0

Inputs: Landmark name, city key.
Outputs: Verified image URL.
Data Store: DS-5 (Image Cache).

P5: Persistence — Process 5.0

Inputs: User itinerary payload.
Outputs: Saved trip record ID.
Data Store: DS-4 (Preferences DB).

P6: Serializer — Process 6.0

Inputs: SQLAlchemy model instance.
Outputs: Sanitized Pydantic JSON response.
Target: React Frontend Client.

3. Core Data Stores & Schema Mapping

STORE ID	ENTITY NAME	KEY SCHEMA FIELDS	ACCESS PATTERN
DS-1	Users Table	`id, email, hashed_password, created_at`	Read/Write on Auth
DS-2	Destinations Table	`id, name, country, hero_image, mood, budget`	Read-heavy, Cached
DS-3	Itineraries Table	`id, user_id, dest_id, days_data (JSONB)`	Write on Gen, Read on Load
DS-4	Preferences Table	`id, user_id, diet, persona, accessibility`	Read on Engine Invoke
DS-5	Media Cache Store	`landmark_name, image_url, verified_flag`	Key-Value Read